Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 "Компьютерные науки и прикладная математика"

Кафедра №806 "Вычислительная математика и программирование"

Лабораторная работа №2 по курсу «Операционные системы»

Группа: М8О-108БВ-25

Студент: Басаргин М.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 25.10.26

Москва, 2026

Постановка задачи

Цель работы: приобретение практических навыков в управлении потоками в ОС и в обеспечении синхронизации между потоками.

Задание: составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы. Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков, а получившиеся результаты объяснить.

Вариант 3: отсортировать массив целых чисел при помощи параллельной сортировки слиянием.

Дополнительные требования: не использовать стандартную библиотеку для работы с потоками, только POSIX Threads (Linux и macOS) или Windows API; реализовать две версии алгоритма, последовательную и параллельную; замерить время выполнения обеих версий, причём для параллельной версии для каждого значения числа потоков.

Алгоритм

Сортировка слиянием делит массив пополам, рекурсивно сортирует каждую половину и сливает две отсортированные половины в одну. Половины независимы друг от друга и лежат в разных частях массива, поэтому их можно сортировать одновременно в разных потоках, а синхронизация для самой сортировки не нужна.

Последовательная версия (sort_seq) выполняет эту рекурсию в одном потоке.

Параллельная версия (sort_par) устроена так. Если кусок массива не короче CUT = 1024 элементов и лимит потоков не исчерпан, текущий поток создаёт новый поток для левой половины, сам сортирует правую, ждёт левую через pthread_join и сливает результат. Иначе кусок сортируется последовательно. Порог CUT нужен, чтобы не создавать поток ради нескольких элементов: стоимость создания потока больше, чем польза.

Ограничение числа потоков. Ключ -t задаёт максимум одновременно работающих потоков p (главный поток считается). Для этого используется счётчик свободных «слотов» free_slots = p – 1. Перед созданием потока функция take_slot под мьютексом проверяет, что слот есть, и занимает его. После join функция give_slot возвращает слот. Так одновременно работает не больше p потоков. Счётчики общие, поэтому доступ к ним защищён мьютексом, иначе возникла бы гонка данных. Переменная peak хранит максимум одновременно работавших потоков и выводится в конце.

Слияние (merge) идёт через вспомогательный буфер t того же размера, что и массив. Потоки работают с непересекающимися кусками массива и буфера, поэтому данные между ними не пересекаются.

Что сортируется. Сортируется массив из n случайных чисел типа int, созданных функцией rand() (размер n задаётся ключом -n, зерно генератора фиксировано, поэтому при каждом запуске данные одни и те же). Для обеих версий используется одна и та же копия данных.

Проверка корректности. Одни и те же случайные данные копируются в два массива. Один сортируется последовательной версией, другой параллельной, после чего результаты сравниваются между собой, а также проверяется, что массив неубывающий.

Метрики: ускорение и эффективность

Ускорение показывает, во сколько раз параллельная версия быстрее последовательной:

$$S(p) = Ts / Tp(p),$$

где Ts – время последовательной версии, Tp(p) – время параллельной версии на p потоках. При $S > 1$ параллельная версия быстрее, при $S < 1$ медленнее. Идеальное ускорение равно p.

Эффективность показывает, какую долю возможностей p потоков удалось использовать:

$$E(p) = S(p) / p = Ts / (p \cdot Tp(p)),$$

где p – число потоков. Идеальная эффективность равна 1 (100 %). Чем она ниже, тем больше потоки простаивают, ждут друг друга или тратят время на собственное обслуживание (создание, переключение контекста).

Время замеряется с помощью clock_gettime в миллисекундах. Для каждого значения числа потоков эксперимент повторяется несколько раз, в таблицу заносится среднее значение. Ts измеряется на тех же данных в том же запуске.

Код программы

msort.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <unistd.h>
#include <time.h>
#include <pthread.h>

#define CUT 1024

static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static int free_slots;
static int run = 1;
static int peak = 1;
static bool demo = false;
static bool show = false;
static _Thread_local bool slept = false;

static bool take_slot(void)
{
    pthread_mutex_lock(&m);
    bool ok = free_slots > 0;
    if (ok) {
        free_slots--;
        run++;
        if (run > peak) peak = run;
    }
    pthread_mutex_unlock(&m);
    return ok;
}

static void give_slot(void)
{
    pthread_mutex_lock(&m);
    free_slots++;
    run--;
    pthread_mutex_unlock(&m);
}

static void merge(int *a, int *t, size_t n, size_t mid)
```

```
{
    size_t i = 0, j = mid, k = 0;
    while (i < mid && j < n)
        t[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < mid) t[k++] = a[i++];
    while (j < n)   t[k++] = a[j++];
    memcpy(a, t, n * sizeof(int));
}

static void sort_seq(int *a, int *t, size_t n)
{
    if (n < 2) return;
    size_t mid = n / 2;
    sort_seq(a, t, mid);
    sort_seq(a + mid, t + mid, n - mid);
    merge(a, t, n, mid);
}

typedef struct { int *a; int *t; size_t n; } Task;

static void sort_par(int *a, int *t, size_t n);

static void *sort_thread(void *p)
{
    Task *k = (Task *)p;
    sort_par(k->a, k->t, k->n);
    return NULL;
}

static void sort_par(int *a, int *t, size_t n)
{
    if (n < 2) return;
    size_t mid = n / 2;

    if (n >= CUT && take_slot()) {
        Task k = {a, t, mid};
        pthread_t th;
        if (pthread_create(&th, NULL, sort_thread, &k) == 0) {
            sort_par(a + mid, t + mid, n - mid);
            pthread_join(th, NULL);
            give_slot();
        } else {
```

```
            give_slot();
            sort_seq(a, t, n);
            return;
        }
        merge(a, t, n, mid);
    } else {
        if (demo && !slept) {
            slept = true;
            sleep(5);
        }
        sort_seq(a, t, n);
    }
}

static double now_ms(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}

static void print_arr(const char *title, const int *a, long n)
{
    long k = n < 10 ? n : 10;
    printf("%s\n  начало:", title);
    for (long i = 0; i < k; i++) printf(" %d", a[i]);
    printf("\n  конец:  ");
    for (long i = n - k; i < n; i++) printf(" %d", a[i]);
    printf("\n");
}

int main(int argc, char **argv)
{
    long n = 10000000;
    int th = 1;

    int o;
    while ((o = getopt(argc, argv, "n:t:dp")) != -1) {
        if (o == 'n') n = atol(optarg);
        else if (o == 't') th = atoi(optarg);
        else if (o == 'd') demo = true;
        else if (o == 'p') show = true;
```

```
    else return 1;
}
if (th < 1) th = 1;
free_slots = th - 1;

int *a = malloc(n * sizeof(int));
int *b = malloc(n * sizeof(int));
int *t = malloc(n * sizeof(int));
if (!a || !b || !t) {
    fprintf(stderr, "нет памяти\n");
    return 1;
}
srand(1);
for (long i = 0; i < n; i++)
    a[i] = b[i] = rand();

printf("PID=%d, логических ядер=%ld, n=%ld, потоков (лимит)=%d\n",
       getpid(), sysconf(_SC_NPROCESSORS_ONLN), n, th);

if (show) print_arr("Массив до сортировки:", a, n);

double t0 = now_ms();
sort_seq(b, t, n);
double ts = now_ms() - t0;

t0 = now_ms();
sort_par(a, t, n);
double tp = now_ms() - t0;

if (show) print_arr("Массив после параллельной сортировки:", a, n);

bool ok = memcmp(a, b, n * sizeof(int)) == 0;
for (long i = 1; i < n && ok; i++)
    if (a[i - 1] > a[i]) ok = false;

double sp = ts / tp;
double ef = sp / th;

printf("Проверка: %s, пик потоков: %d\n", ok ? "OK" : "ОШИБКА", peak);
printf("Число потоков | Время исполнения (мс) | Ускорение | Эффективность\n");
printf("%13d | %21.2f | %9.3f | %13.3f   (Ts = %.2f мс)\n", th, tp, sp, ef, ts);
```

```
    free(a); free(b); free(t);
    return 0;
}
```

Сборка: gcc -std=gnu11 -O2 -pthread msort.c -o lab_2. Запуск: ./lab_2 -n 10000000 -t 4, где -n – размер массива, -t – максимум потоков, -d – демонстрационный режим, -p – показать начало и конец массива до и после сортировки.

Протокол работы программы

Среда: процессор Intel Core i7-8565U (4 физических ядра, 8 логических потоков благодаря Hyper-Threading), ОС WSL2 Ubuntu – 24.04.4. Число логических ядер (команда nproc): 8.

Тест 1. Корректность

```
$ ./lab_2 -n 10000000 -t 4
PID=47685, логических ядер=8, n=10000000, потоков (лимит)=4
Проверка: OK, пик потоков: 4
Число потоков | Время исполнения (мс) | Ускорение | Эффективность
            4 |                541.81 |     3.367 |         0.842   (Ts = 1824.44 мс)
```

Проверка «OK» означает, что результат параллельной версии совпал с последовательной и массив отсортирован. Пик потоков равен заданному лимиту. Результат одного запуска (S = 3,367) выше среднего по пяти запускам из таблицы 1 (S = 2,245): разбиение массива между потоками зависит от того, какой поток успеет первым занять свободный слот, поэтому время от запуска к запуску меняется.

Тест 2. Число потоков средствами ОС

Программа запущена в демонстрационном режиме (-d), при котором каждый поток 5 секунд ждёт, чтобы все потоки были живы одновременно. В это время средствами ОС получено число потоков процесса (PID подставляется автоматически через $!).

```
$ ./lab_2 -n 2000000 -t 8 -d & sleep 2; ps -o nlwp -p $!; grep Threads /proc/$!/status; ps
-T -p $!; wait
[1] 47836
PID=47836, логических ядер=8, n=2000000, потоков (лимит)=8
NLWP
   8
Threads:        8
   PID    SPID TTY          TIME CMD
  47836   47836 pts/0    00:00:00 lab_2
  47836   47838 pts/0    00:00:00 lab_2
  47836   47839 pts/0    00:00:00 lab_2
  47836   47840 pts/0    00:00:00 lab_2
  47836   47841 pts/0    00:00:00 lab_2
  47836   47842 pts/0    00:00:00 lab_2
  47836   47843 pts/0    00:00:00 lab_2
  47836   47844 pts/0    00:00:00 lab_2
Проверка: OK, пик потоков: 8
```

```
ads /proc/$!/status; ps -T -p $!; wait
[1] 47836
PID=47836, логических ядер=8, n=2000000, потоков (лимит)=8
NLWP
   8
Threads:        8
    PID    SPID TTY          TIME CMD
  47836   47836 pts/0    00:00:00 lab_2
  47836   47838 pts/0    00:00:00 lab_2
  47836   47839 pts/0    00:00:00 lab_2
  47836   47840 pts/0    00:00:00 lab_2
  47836   47841 pts/0    00:00:00 lab_2
  47836   47842 pts/0    00:00:00 lab_2
  47836   47843 pts/0    00:00:00 lab_2
  47836   47844 pts/0    00:00:00 lab_2
Проверка: OK, пик потоков: 8
Число потоков | Время исполнения (мс) | Ускорение | Эффективность
            8 |               5417.52 |     0.113 |         0.014   (Ts = 612.50 мс)
[1]+  Done                    ./lab_2 -n 2000000 -t 8 -d
```

Рис. Определение числа потоков средствами ОС в WSL.

Команда ps -o nlwp показывает число потоков процесса (NLWP = 8), строка Threads в файле /proc/<PID>/status тоже содержит 8, а ps -T выводит каждый поток отдельной строкой: у всех один PID и разные номера потоков (SPID). Это 7 созданных потоков и главный, то есть столько, сколько задано ключом -t 8. Значение совпадает с «пиком потоков», который печатает сама программа. Время в этом режиме не используется, так как оно включает паузы.

Тест 3. Что именно сортируется

Ключ -p выводит по 10 первых и последних элементов массива до и после параллельной сортировки (вне замеров времени).

```
user@WIN-R5HR0RLRAAJ:/mnt/c/users/user/desktop/lab/OC$ ./Msort -n 10000000 -t 4 -p
PID=48976, логических ядер=8, n=10000000, потоков (лимит)=4
Массив до сортировки:
  начало: 1804289383 846930886 1681692777 1714636915 1957747793 424238335 719885386 1649760492 596516649 1189641421
  конец:   1643046783 1795098895 630162568 1031706584 607469124 1097364974 624978040 681519524 601197907 186687031
Массив после параллельной сортировки:
  начало: 37 95 614 858 880 1210 1429 2126 2172 2218
  конец:   2147481500 2147481509 2147481553 2147481902 2147481979 2147482312 2147482459 2147482567 2147482751 2147483025
Проверка: OK, пик потоков: 4
Число потоков | Время исполнения (мс) | Ускорение | Эффективность
            4 |                869.38 |     2.016 |         0.504   (Ts = 1752.32 мс)
```

Рис. Вывод команды ./lab_2 -n 10000000 -t 4 -p в WSL.

Исследование ускорения и эффективности

Все измерения проведены на массиве из n = 10 000 000 случайных целых чисел типа int, каждое значение – среднее по 5 запускам. Процессор Intel Core i7-8565U: 4 физических ядра, 8 логических (nproc = 8). Время последовательной версии на тех же данных Ts ≈ 1680 мс.

Зависимость от числа потоков

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	1739,29	0,996	0,996
2	839,04	1,975	0,987
3	866,50	1,911	0,637
4	764,28	2,245	0,561
8 (= числу ядер)	547,38	3,075	0,384
16	524,14	3,251	0,203
128	468,60	3,564	0,028
1024	592,48	2,770	0,003

Таблица 1. Время, ускорение и эффективность для n = 10 000 000. Потоков меньше числа ядер: 1, 2, 3, 4; равно числу ядер: 8; сильно больше: 16, 128, 1024.

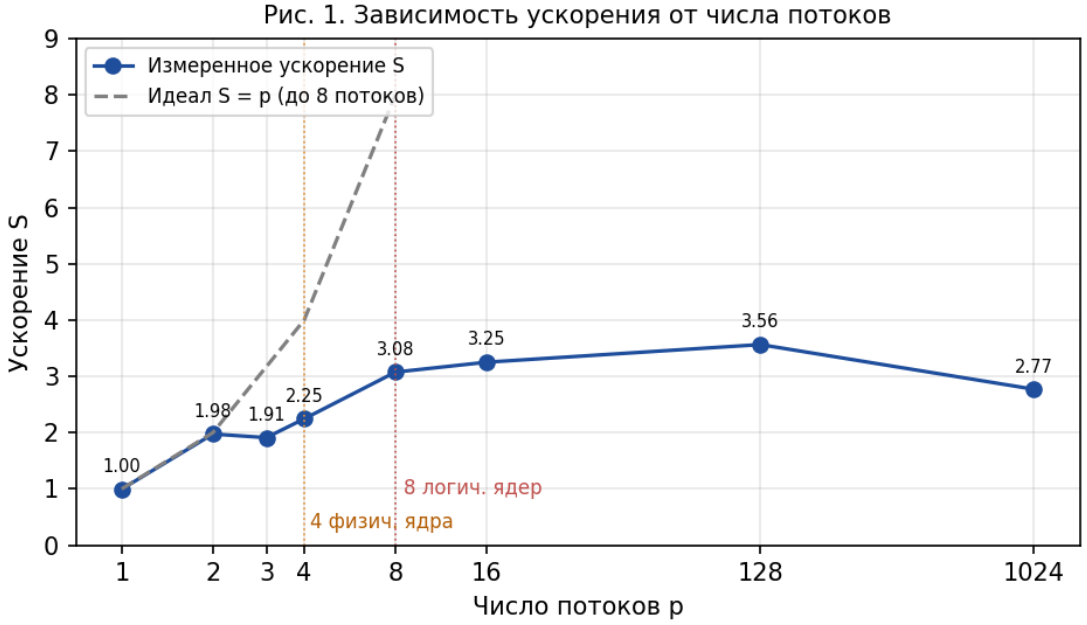


Рис. 1. Зависимость ускорения от числа потоков

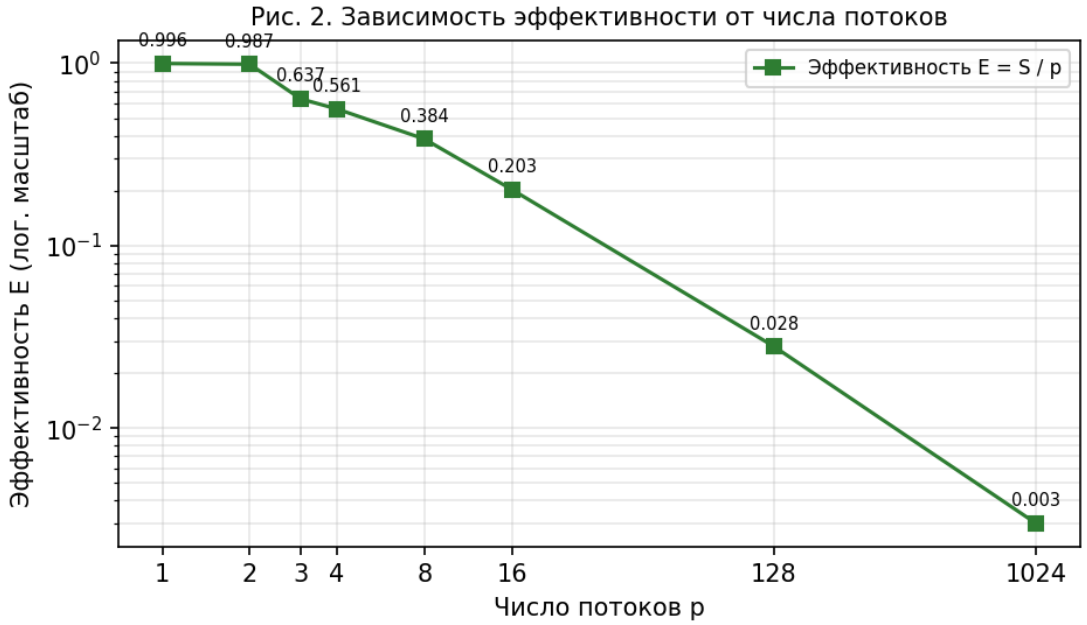


Рис. 2. Зависимость эффективности от числа потоков

Зависимость от размера входных данных

Размер массива n	Ts (мс)	Tp (мс)	Ускорение	Эффективность
100 000	21,4	7,88	2,720	0,340
1 000 000	138,3	48,60	2,845	0,356
10 000 000	1683	547,38	3,075	0,384

Таблица 2. Зависимость от размера массива при p = 8 (равно числу логических ядер). Ts получено как Tp · S из вывода программы.

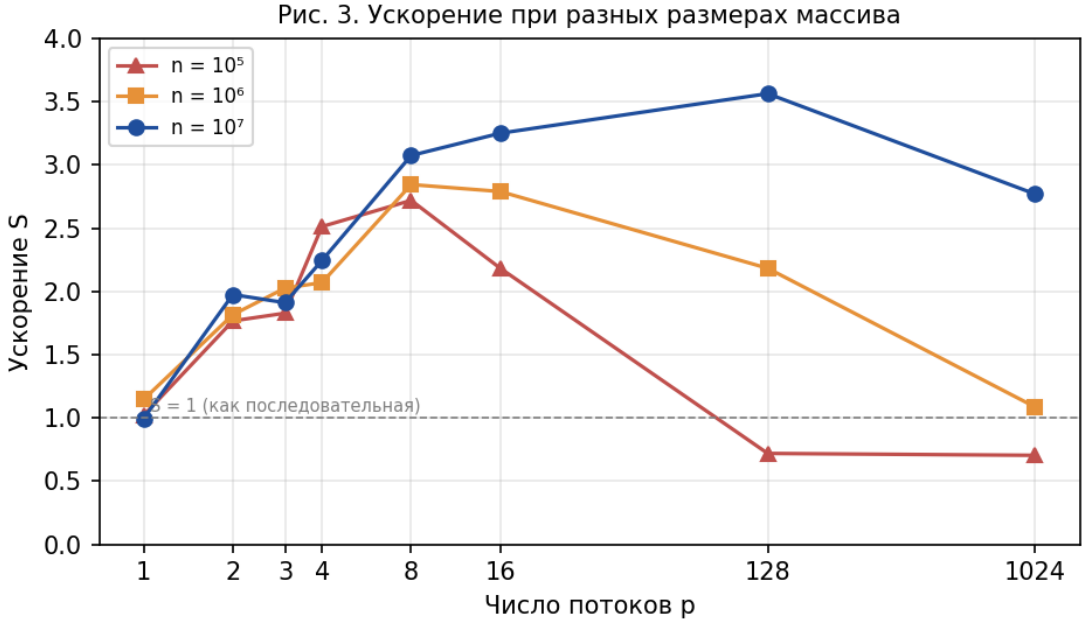


Рис. 3. Ускорение при разных размерах массива

Время при n = 10^5 и n = 10^6 получено теми же замерами, что в таблице 1 (по 5 запусков на каждое число потоков). Ts в разных строках одной серии отличается: при n = 10^6 от 139 до 160 мс. Причина – шум на ноутбуке (WSL, изменение частоты процессора), поэтому разницу в ускорении меньше 10 % нельзя считать значимой. Повторная серия при n = 10^6 дала для p = 8 ускорение 2,960, то есть в пределах шума.

Вывод

В этой лабораторной работе была реализована последовательная и параллельная сортировка слиянием на POSIX Threads, ограничено число одновременно работающих потоков ключом запуска и показано число потоков средствами ОС (ps, top, /proc). Синхронизация понадобилась для общих счётчиков потоков: чтобы они не портились при одновременном изменении, доступ к ним защищён мьютексом, а pthread_join заставляет поток дождаться готовой половины перед слиянием.

Объяснение результатов. При p = 1 ускорение равно 0,996, то есть параллельная версия с одним потоком работает как последовательная, а накладные расходы на проверку слотов ничтожны. При p = 2 ускорение 1,975 при эффективности 0,987: две половины массива сортируются одновременно, и результат почти идеальный. При p = 3 время (866,50 мс) не лучше, чем при p = 2 (839,04 мс). Причина в том, что массив делится пополам, и три потока не могут разделить работу поровну: получаются куски в n/2, n/4 и n/4 элементов, и самый большой кусок задаёт время работы. При p = 4 ускорение вырастает до 2,245, так как куски получаются меньше и их больше. Точное разбиение массива между потоками зависит от того, какой поток успеет первым занять свободный слот, поэтому оно может немного меняться от запуска к запуску, и я усреднял время по пяти запускам.

Ускорение заметно меньше идеального S = p (при p = 4 получилось 2,245 вместо 4, при p = 8 – 3,075 вместо 8). Во-первых, часть работы остаётся последовательной: последнее слияние двух половин по n элементов выполняет один поток, предыдущее – два потока и так далее, поэтому ускорение ограничено (закон Амдала). Во-вторых, сортировка слиянием много читает и пишет память, а потоки делят шину памяти и кэш, поэтому быстрее процессор не делает. В-третьих, у процессора только 4 физических ядра: 8 логических потоков получаются за счёт Hyper-Threading, при котором два потока делят одно ядро (его блоки вычислений и кэш), поэтому они дают гораздо меньше, чем два полноценных ядра.

При p = 4, равном числу физических ядер, ускорение 2,245, а при p = 8, равном числу логических ядер, – 3,075. Удвоение потоков за счёт Hyper-Threading даёт прирост лишь примерно в 1,4 раза, а не в 2. Значения при p = 16 (3,251) и p = 128 (3,564) немного выше, хотя ядер больше нет: более мелкие куски лучше выравнивают нагрузку между ядрами, так как поток, закончивший быстро, освобождает ядро для других. Выше потолка, который задают последовательные слияния и память, ускорение не поднимается. При p = 1024 ускорение падает до 2,770: ОС тратит время на создание и переключение сотен потоков, каждый из которых занимает память под стек, и это перевешивает выигрыш.

Эффективность E = S / p монотонно падает с ростом числа потоков: от 0,996 при p = 1 до 0,003 при p = 1024. Ускорение растёт гораздо медленнее, чем число потоков, поэтому на каждый поток приходится всё меньше полезной работы. Лучшая эффективность у малых p (1 и 2), а лучшее время получено при p = 128, хотя этот выигрыш над p = 8 небольшой (469 мс против 547 мс) и обходится в десятки раз большей загрузкой потоков.

Зависимость от размера данных. При p = 8 ускорение растёт с размером массива: 2,72 при n = 10^5, 2,85 при n = 10^6 и 3,08 при n = 10^7. Чем меньше массив, тем заметнее

накладные расходы на создание потоков по сравнению с самой сортировкой. Это особенно видно при большом числе потоков: при n = 10^5 и p = 128 или 1024 параллельная версия медленнее последовательной (S = 0,72 и 0,70), потому что на создание сотен потоков для массива в 100 тысяч элементов уходит больше времени, чем на сортировку. При n = 10^6 и p = 1024 ускорение всего 1,09. Пик потоков при этом 641, а не 1024: куски короче 1024 элементов потоки уже не получают. Для малых массивов лучшее число потоков равно числу логических ядер (p = 8), а для самого большого массива (n = 10^7) небольшой выигрыш остаётся и при p = 16 и 128.

Практически число потоков нужно брать близким к числу логических ядер, и лучше степенью двойки: меньшее значение недоиспользует процессор, а намного большее почти ничего не даёт и при очень больших значениях замедляет программу.